

NTRO Dark Web Threat Actor De-Anonymization System

Smart India Hackathon (SIH26151) | Complete Technical Documentation (Steps 1 - 8)

System Overview: This platform is an automated end-to-end intelligence and forensic attribution engine built for intelligence analysts to de-anonymize threat actors operating on Tor hidden services (.onion). The architecture spans raw crawling, deterministic entity resolution, infrastructure fingerprinting, NLP stylometry, confidence scoring, real-time query dashboards, and evidentiary reporting.

Step	Module Name	Target Objective & Core Mechanism
Step 1	Evidence Collection	Scrapes BTC, ETH, XMR addresses, handles, and PGP blocks; logs SHA-256 hashes.
Step 2	Storage & Resolution	Relational SQLite schema; multi-pivot merging into unified Threat Actor UUIDs.
Step 3	Correlation Engine	Bipartite graph mapping (NetworkX) detecting co-conspirator syndicates & multi-hop paths.
Step 4	OPSEC & Infra Analysis	Murmur3 favicon hashing & DOM tree skeleton analysis exposing shared hidden servers.
Step 5	AI Stylometric Analysis	Character n-gram TF-IDF & Cosine Similarity correlating forum writing patterns.
Step 6	Attribution & Scoring	Weighted heuristic engine scoring overall forensic linkage confidence (0 - 100).
Step 7	Analyst Dashboard	Interactive browser workspace with search, target inspection, and entity graph links.
Step 8	Forensic Case Export	Court-admissible PDF dossier, JSON payload, and CSV export with full chain of custody.

Step 1: Evidence Collection (entity_extractor.py)

Parses unstructured HTML payloads from darknet posts using regular expressions. Extracts Legacy and Bech32 Bitcoin, Ethereum, standard and sub-address Monero, PGP public key blocks, and communications handles (Jabber, Telegram, TOX). To maintain court admissibility, each ingested payload generates an immutable SHA-256 checksum and UTC timestamp.

Step 2: Evidence Storage & Deterministic Resolution (resolution_pipeline.py)

Manages the relational intelligence.db SQLite schema. Threat actors cycle aliases across forums (e.g., DreadOps vs. ApexBreach). The resolution pipeline executes multi-pivot lookups across crypto wallets, contact handles, and PGP blocks. When a matching identifier is found, the engine binds the new identity to the target's existing master UUID instead of creating duplicates.

Step 3: Evidence Correlation Engine (correlation_engine.py)

Models threat data as an undirected bipartite graph using NetworkX, connecting threat actors to shared financial and communication identifiers. Implements connected components analysis to identify criminal rings and shortest-path graph traversals to establish multi-hop forensic chains linking actors who have not directly communicated.

Step 4: OPSEC & Infrastructure Analysis (opsec_analyzer.py)

Identifies backend operational security leaks across hidden services without relying on textual data. Computes Shodan-compatible 32-bit MurmurHash3 signatures from base64 favicon bytes and extracts SHA-256 DOM skeleton tag trees. Identical hashes across distinct .onion addresses prove shared site administration or clone kit reuse.

Step 5: AI-Based Stylometric Analysis (stylometry_analyzer.py)

Performs linguistic profiling on anonymous forum text. Extracts lexical features (sentence length, punctuation density, capitalization ratios) and builds Character n-gram TF-IDF vector embeddings (3-5 char ranges). Computes Cosine Similarity to identify syntax habits, spelling patterns, and idiosyncratic typing styles across aliases.

Step 6: Attribution & Confidence Scoring Engine (confidence_scorer.py)

Aggregates disparate findings into an auditable confidence score (0 - 100): PGP Match (+40), Crypto Address (+35), Direct Handle (+25), Infrastructure/Favicon Match (+20), and Stylometric Match (+20). Updates threat_actors.risk_score and generates a human-readable audit trail explaining the evidence weight.

Step 7: Investigation Dashboard (api.py)

A unified FastAPI web workspace accessible at /dashboard. Allows investigators to search threat actors, view risk indicators, inspect unified cross-market aliases and crypto trails, examine raw forensic hashes, and download reports with one click.

Step 8: Report Generation & Export (report_generator.py)

Pulls the complete target case file and exports structured JSON, CSV tables, and tamper-proof PDF dossiers containing case headers, alias timelines, financial footprint tables, and forensic chain-of-custody checksums.

System Operational Runbook (Execution Commands)

Action	PowerShell Command
1. Initialize DB & Seed Data	python db_manager.py ; python main.py
2. Run Graph Correlation	python correlation_engine.py
3. Run OPSEC & Infra Check	python opsec_analyzer.py
4. Run Stylometry & Scoring	python stylometry_analyzer.py ; python confidence_scorer.py
5. Export Reports (PDF/CSV)	python report_generator.py
6. Launch Analyst Dashboard	uvicorn api:app --reload --> http://127.0.0.1:8000/dashboard